

AOA Analysis of Algorithm

{ (Unit 1) Fundamental & Devide
& Conquer method
(evaluate an algo) }

* Algorithm :- Set of steps of ~~processing~~ operations to solve a problem performing calculation, data processing & automated reasoning tasks.

- It should be efficient method.
- Expressed within finite time & space for large number of inputs.
- Algorithm is platform & programming language independent.

Ex. Sum of two numbers

Algorithm

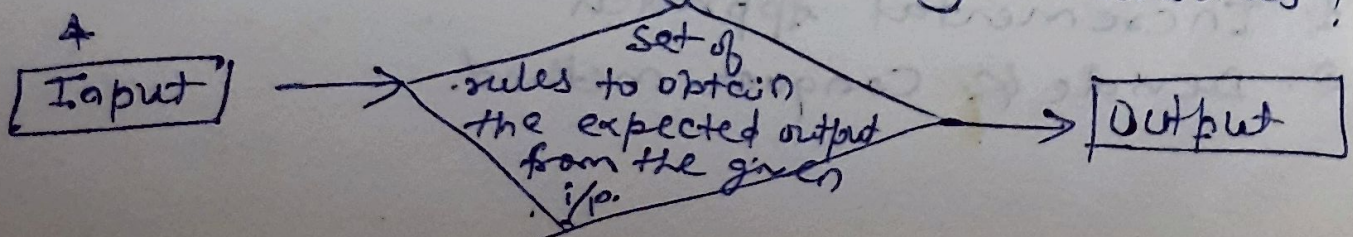
1. Start
2. Input A & B
3. $Sum = A + B$
4. print Sum
5. end.

Note Algorithm must have atleast one output.

→ It can have zero or more inputs.

while evaluating an Algo. we should check-

1. Time → in computation duration.
2. Space → How many memory it consumes?



* Difference between Algorithm, pseudocode & program

Algorithm	Pseudocode	Program
- logical step-by-step solution	- Algorithm ke code-like representation	- Actual executable program
- Language independent	- Mostly language independent	- Language specific
- Human-readable	- More structured	- Compiler/interpreter required
- Focuses on logic	- Focus on logic + structure	- Focus on implementation
Ex - Step 1 Step 2	INPUT A DISPLAY A	scanf("%d", &a)

* Characteristics of an Algorithm

1. well defined input
2. well defined output
3. Clear & unambiguous
4. Finite steps
5. Feasible
6. Language independent.

Algorithm Design

Develop an appropriate strategy to solve a problem.

It focuses on to minimize time & space complexity.

Two Basic Approaches -

1. Incremental Approach
2. Divide & Conquer method.

(a) Incremental Approach:-

- Top-down Approach.

- Proceed from start to compare element iteratively & finally get the solution.

Ex = Bubble sort
Insertion sort
Selection sort

1) Bubble sort:-

It repeatedly compares two adjacent elements.

If they are in wrong order swap them.

After one complete pass, largest element moves at last. (end).

Ex =

8	5	3	1	4	2	7
---	---	---	---	---	---	---

Compare

$8 > 5$

Swap

5 | 8 | 3 | 1 | 4 | 2 | 7

$8 > 3$ swap 5 3 8 1 4 2 7

$8 > 1$ swap 5 3 1 8 4 2 7

$8 > 4$ swap 5 3 1 4 8 2 7

$8 > 2$ swap 5 3 1 4 2 8 7

$8 > 7$ swap

5	3	1	4	2	7	8
---	---	---	---	---	---	---

After 1st pass

2nd pass

5 3 1 4 2 7 8

5 > 3 swap 3 5 1 4 2 7 8

5 > 1 swap 3 1 5 4 2 7 8

5 > 4 swap 3 1 4 5 2 7 8

5 > 2 swap 3 1 4 2 5 7 8

3rd pass

3 1 4 2 5 7 8

3 > 1 swap 1 3 4 2 5 7 8

3 > 4 swap 1 3 2 4 5 7 8

4th pass

1 3 2 4 5 7 8

3 > 2 swap 1 2 3 4 5 7 8

done

Algorithm

1. Start
2. Repeat for $i = 0$ to $n-2$
3. Repeat for $j = 0$ to $n-i-2$
4. if $A[j] > A[j+1]$
5. swap $A[j]$ & $A[j+1]$
6. End If
7. End Repeat
8. End Repeat
9. Stop.

Complexity

space = $O(1)$

time = $O(n^2)$

pseudo code

```
for i = 0 to n-2
  for j = 0 to n-i-2
    if A[j] > A[j+1]
      temp = A[j]
      A[j] = A[j+1]
      A[j+1] = temp
```


Code (In Java)

```
#include <ostream>
using namespace
```

```
import java.util.Arrays;
```

```
public class main {
```

```
    public static void bubbleSort(int[] arr,
                                   int n) {
```

```
        for (int i = 0; i < n-1; i++)
```

```
        {
```

```
            for (int j = 0; j < n-1-i; j++)
```

```
            {
```

```
                if (arr[j] > arr[j+1])
```

```
                {
```

```
                    int temp = arr[j];
```

```
                    arr[j] = arr[j+1];
```

```
                    arr[j+1] = temp;
```

```
                }
```

```
            }
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int[] arr = { 5, 2, 8, 1, 3};
```

```
        int n = arr.length;
```

```
        bubbleSort(arr, n);
```

```
        System.out.print("Sorted Array");
```

```
        for (int i = 0; i < n; i++) {
```

```
            System.out.print(arr[i] + " ");
```

```
        }
```

```
    }
```

```
}
```


2. Selection Sort :-

Find smallest element put it in its correct position

→ So in first pass, smallest element would be at first position.

Remaining unsorted part, smallest ~~for~~ will be at start.

Ex 5 3 8 1 2 4

we will have sorted & unsorted part

[] | [5, 3, 8, 1, 2, 4]

phase 1 find smallest = 1

[1] | [5, 3, 8, 2, 4]

phase 2 smallest = 2

[1, 2] | [5, 3, 8, 4]

phase 3 smallest = 3

[1, 2, 3] | [5, 4, 8]

phase 4 smallest = 4

[1, 2, 3, 4] | [5, 8]

phase 5 smallest

[1, 2, 3, 4, 5] | [8]

[1, 2, 3, 4, 5, 8] Ans

Ex: [5 3 8 1 4 2]

Smallest = 1

pass 1 swap 5 $\leftrightarrow$ 1 1 3 8 5 4 2

Smallest 2

pass 2 swap 3 $\leftrightarrow$ 2 1 2 8 5 4 3

Smallest 3

pass 3 swap 8 $\leftrightarrow$ 3 1 2 3 5 4 8

Smallest = 4

pass 4 swap 5 $\leftrightarrow$ 4 1 2 3 4 5 8

Sorted

Algorithm

1. Start
2. Set $i = 0$
3. Repeat Steps 4-8 for $i = 0$ to $n-2$
4. Set $\text{minIndex} = i$
5. Repeat for $j = i+1$ to $n-1$
6. if $A[j] < A[\text{minIndex}]$, then set $\text{minIndex} = j$
7. Swap $A[i]$ & $A[\text{minIndex}]$
8. Increment i
9. Repeat until $i = n-1$
10. Stop

Time complexity $O(n^2)$

Space complexity = $O(1)$

Code in Java

```
public class Main {  
    public static void selectionSort(  
        int[] arr) {  
        int n = arr.length;  
        for (int i = 0; i < n - 1; i++) {  
            int minIndex = i;  
            for (int j = i + 1; j < n; j++) {  
                if (arr[j] < arr[minIndex])  
                {  
                    minIndex = j;  
                }  
            }  
            int temp = arr[minIndex];  
            arr[minIndex] = arr[j];  
            arr[i] = temp;  
        }  
    }  
}
```

```
public static void main(String[] args) {  
    int[] arr = {5, 2, 8, 1, 3};  
    selectionSort(arr);  
    System.out.print("Sorted array");  
    for (int val : arr) {  
        System.out.print(val + " ");  
    }  
}
```

Complexity

Time = $O(n^2)$

Space = $O(1)$

Insertion Sort :-

Take one element from the unsorted part & insert it into its correct position in the already sorted part.

Ex: 5 3 8 1 4 2

Pick 5

5 | 3 8 1 4 2
 key

pick 3 compare from sorted

$5 > 3$ No shift

3 5 | 8 1 4 2

pick 8 compare from sorted

$5 < 8$ No keep it as it is

3 5 8 | 1 4 2

pick 1 compare from sorted

$8 > 1$ then shift

Now next $5 > 1$ swap

Now Next $3 > 1$ shift

1 3 5 8 | 4 2

pick 4 compare from sorted

$8 > 4$ shift

$5 > 4$ shift

$3 > 4$ No, keep it there

1 3 4 5 8 | 2

1 3 4 5 8 | 2

pick 2, compare from sorted

8 > 2 shift

5 > 2 shift

4 > 2 shift

3 > 2 shift

1 > 2 No, keep as it is

1 2 3 4 5 8

Done

Algorithm

1. Start
2. Consider the first element as sorted
3. for $i = 1$ to $n-1$:
 sort $AC[i]$ in key
 set $j = i-1$
 compare key with element before it
 while $j = 0$ & $AC[j] > \text{key}$:
 shift $AC[j]$ one position to right
 decrement j .
 insert key at $AC[j+1]$
4. Repeat until all element are processed.
5. stop.

Time Complexity

worst case = $O(n^2)$

best case = $O(n)$

Space Complexity

$O(1)$

• Java Code for Insertion Sort

```
public class Main {  
    public static void insertionSort(int[] arr) {  
        int n = arr.length;
```

```
        for (int i = 1; i < n; i++) {  
            int key = arr[i];  
            int j = i - 1;
```

```
            while (j >= 0 && arr[j] > key) {  
                arr[j+1] = arr[j];  
                j = j - 1;
```

```
            }
```

```
            arr[j+1] = key;
```

```
        }
```

```
    }
```

```
    public static void main(String[] args) {
```

```
        int[] arr = {5, 2, 8, 1, 3};
```

```
        insertionSort(arr);
```

```
        System.out.print("Sorted Array");
```

```
        for (int val : arr) {
```

```
            System.out.print(val + " ");
```

```
        }
```

```
    }
```

```
}
```


* Algorithm Analysis :-

Analysis of Algorithm based on two main ways

(i) Input size (n) * efficiency.

(ii) Running Time

Running Time $\propto n$

• Space Complexity

The amount of space required by an algorithm or program.

→ It depends upon input size (n)

• Time Complexity

Amount of time taken by an algorithm in its complete execution.

→ Depends on input size

Ex: $O(1)$, $O(\log n)$, $O(n^2)$, $O(n)$ etc

Note Time Complexity only care about growth of time as n increase. Not on machine/device execution time.

Note ① Constant Complexity → $O(1)$

only one or constant number of operation.

② Linear Complexity → $O(n)$

Ex: iteration, single loop.

③ Quadratic Complexity → $O(n^2)$

Ex: Nested loop. $n \times n = n^2$

④ logarithmic Complexity → $O(\log(n))$

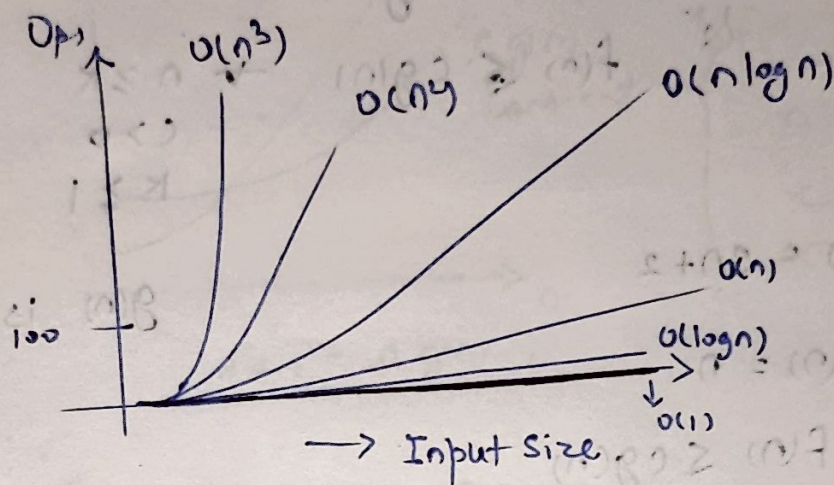
⑤ exponential Complexity. 2^n

• Order of growth

Order of growth tells us about how the amount of work of an algorithm increase as the input size n increases.

*** Imp.

$$O(1) < O(\log n) < O(n) < O(n \log n) < O(n^2) < O(n^3) < O(2^n) < O(n!)$$



* Asymptotic Notation :-

Mathematical Notation used to describe the run time of an algorithm when the input tends towards a particular value or a limiting value.

There are mainly 3 asymptotic Notations :-

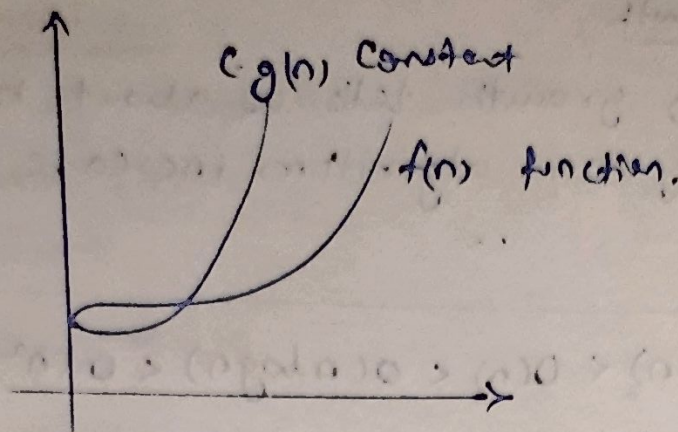
- Big-O Notation
- Omega Notation
- Theta Notation

1) Big-O Notation :-

used to define upper bound of an algorithm

→ Indicate maximum time required by an algorithm for all input values.

→ Describes the worst case of an algorithm.



Ex

$$f(n) = O(g(n))$$

$$f(n) \leq c g(n) \quad \forall n \geq k$$

$$c > 0$$

$$k \geq 1$$

Ex

$$f(n) = 3n + 2$$

Soln

$$g(n) = n$$

$g(n)$ is order of $f(n)$

$$f(n) \leq c g(n)$$

$$3n + 2 \leq c(n) \quad \therefore \text{we will check for all inputs}$$

$c = 1$ to until it satisfies

$$c = 4 \quad n \geq 2$$

(upper bound)

Ex

$$f(n) = 3n^2 + 5n + 10$$

$$g(n) = n^2$$

$$f(n) \leq c g(n)$$

$$3n^2 + 5n + 10 \leq c n^2$$

$$3 + 5/n + 10/n^2 \leq c$$

$$\text{on } n = 1$$

$$18 \leq c$$

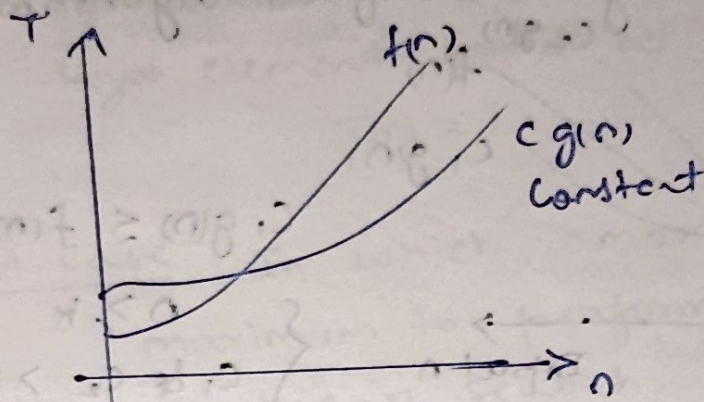
upper bound

2. Big-Ω Notation

used to define the lower bound of an algorithm in terms of T.C.

→ Indicates minimum time required by an algo

→ It describes best case of an algo.



$$\left\{ \begin{array}{l} f(n) \geq c g(n) \\ n \geq k \\ c > 0 \\ k \geq 1 \end{array} \right\}$$

$$f(n) = \Omega(g(n))$$

Ex $f(n) = 3n + 2$ $g(n) = n$

$$f(n) \geq c g(n)$$

$$3n + 2 \geq c(n)$$

$c = 1$ lower bound.
 $n \geq 1$

Ex $f(n) = 3n^2 + 5n + 10$

$$f(n) \geq c g(n)$$

$$g(n) = n^2$$

$$3n^2 + 5n + 10 \geq c n^2$$

$$3 + 5/n + 10/n^2 \geq c \quad \text{for } n \geq 1$$

$$18 \geq c$$

So $n \geq 1$

$c \leq 18$

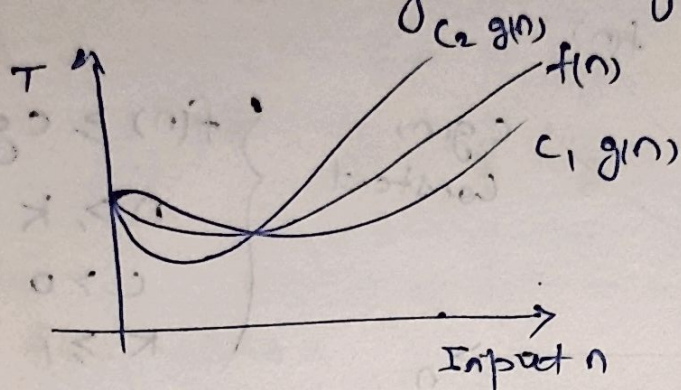
lower bound.

3) Big- Θ Notation:- (tight bound)

Used to define the average bound of an algorithm in terms of TC.

→ It indicates the average time required read by an algorithm.

→ describes average case of an algorithm $\forall c$.



$$c_1 g(n) \leq f(n) \leq c_2 g(n)$$

$$\left\{ \begin{array}{l} n \geq k \\ c_1, c_2 > 0 \\ k \geq 1 \end{array} \right\}$$

$$f(n) = \Theta(g(n))$$

for previous ex. $n = \log$

$$f(n) = 3n + 2$$

$$g(n) = n$$

$$c_1 g(n) \leq f(n) \leq c_2 g(n) \quad \forall c_1, c_2 \geq 0 \text{ \& } k \geq 1$$

$$c_1 n \leq 3n + 2 \leq c_2 n$$

$$\begin{array}{ccc} \downarrow & & \downarrow \\ c_1 = 1 & & c_2 = 4 \end{array}$$

for $n \geq 2$

Note Relationship between Θ , Ω & O

$$\boxed{\Theta(g(n)) = O(g(n)) + \Omega(g(n))}$$

* Space Complexity

* Best, Worst & Average Case

1) Best Case In which algorithm, amount of work is minimum to perform.

Ex In linear search, target element found at first position.

$$T(n) = 1$$

2) Worst Case In which, amount of work is maximum to perform.

Ex In linear search, target element found at last position or it is not in that array.

$$T(n) = n$$

3) Average Case In which, amount of work is avg.
minimum $< T(n) <$ maximum

for sortings	best	worst	Avg.
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$

* Space Complexity Represents space requirement by an algorithm. $S(n)$

for constant variable $O(1)$

for array size $O(n)$

for 2D array $O(n^2)$

for Auxiliary array $O(1)$ (extra memory space)

for sorting algo bubble, insertion, selection sort = $O(1)$

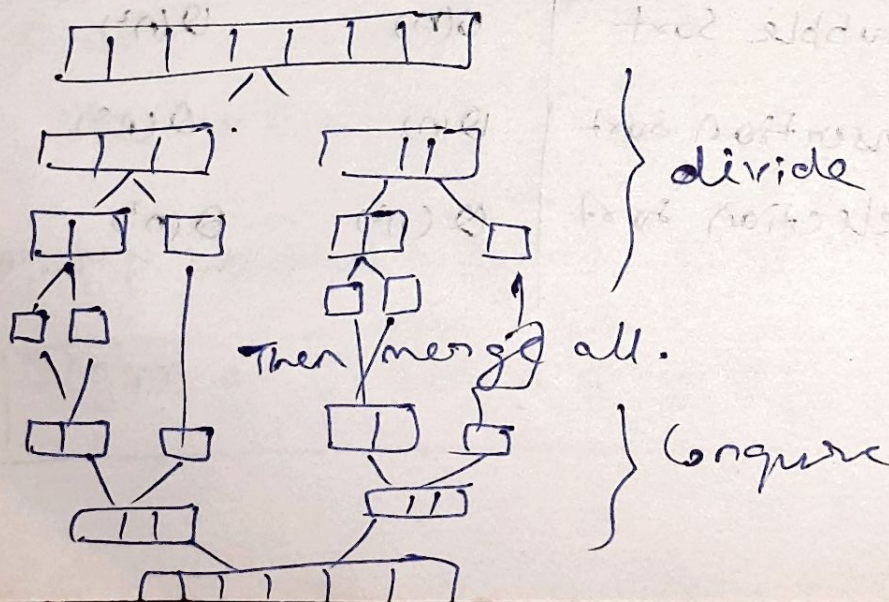
Note for some algo we use more memory to reduce computation time.

Ex In merge sort, recursion is used to reduce T.C. ($O(n)$ space complexity)

(b) Divide & Conquer Approach :-

In this approach, we divide a big problem into sub problem & each sub problem is recursively & combining these solution.

Ex merge, quick & heap sort etc.



Advantage

- 1) Efficient Algorithm
- 2) Naturally recursive
- 3) Breaks natural complex problem
- 4) Parallelism potential
- 5) Useful for large inputs.

Disadvantage

- 1) Recursion Overhead
- 2) Sometimes extra memory
- 3) Combining can be expensive
- 4) Poor split can hurt.

* Binary Search

Finding an element from an array efficiently.

- fastest searching Algo.
- Applies on sorted Array.
- $mid = (low + high) / 2$

Complexity

Best = $O(1)$

Worst = $O(\log n)$

Avg = $O(\log n)$

Algorithm

1. Set $low = 0$

2. Set $high = n - 1$

3. Repeat while $low \leq high$

 Calculate $mid = (low + high) / 2$

 if $A[mid] == key$ return position

 if $key < A[mid]$ set

$high = mid - 1$

 otherwise set

$low = mid + 1$

4. if loop ends, element not found.

5. stop.

$A = [10, 20, 30, 40, 50, 70, 80]$

target = 70

① low = 0 mid = 3
high = 6 $A[3] = 40$

since $40 < 70$
low = mid + 1 = 4

② low = 4 mid = 5
high = 6 $A[5] = 70$ element found
~~low = mid + 1 = 5~~

~~low = 5~~
~~high = 6~~ mid = 5

~~$A[5] = 70$~~

Java Code

```
public class main {
```

```
    public static mainvoid binarySearch (int[] arr,  
        int target) {
```

```
        int low = 0;
```

```
        int high = arr.length - 1
```

```
        while (low <= high) {
```

```
            int mid = low + (high - low) / 2;
```

```
            if (arr[mid] == target) {  
                return mid; }  
            }
```

```
            if (arr[mid] < target) {
```

```
                low = mid + 1; }  
            }
```

```
            if (arr[mid] > target) {
```

```
                High = mid - 1; }  
            }
```

```
}
```

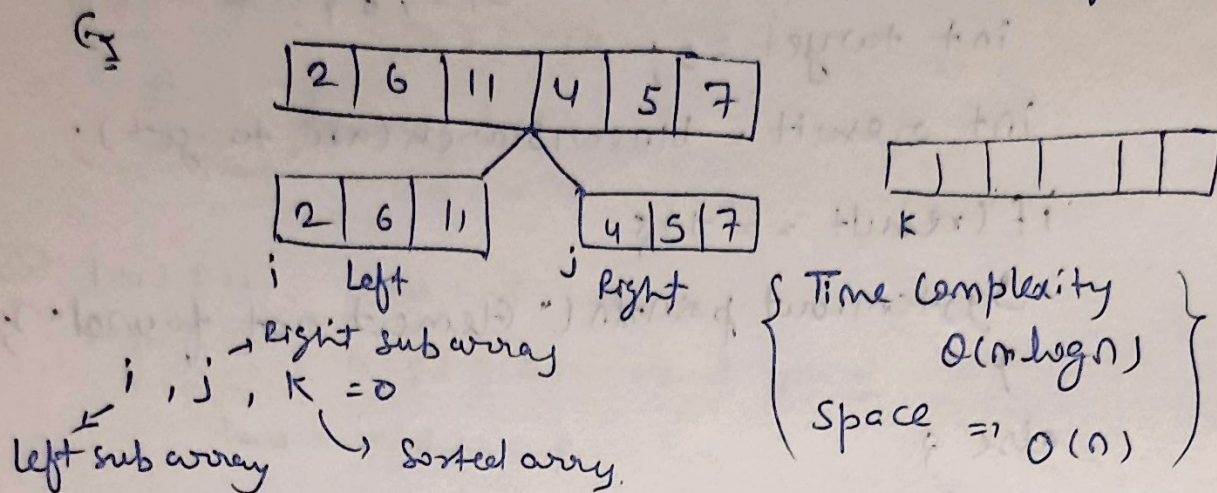


```
    return -1;
}
```

```
public static void main (String[] args) {
    int[] arr = { 1, 2, 3, 5, 8 };
    int target = 5;
    int result = binarySearch(arr, target);
    if (result == -1) {
        System.out.println("Element not found");
    }
    else {
        System.out.println("Element found at " + result);
    }
}
```


* Merge Sort *

Merge Sort is comparison based sorting Algorithm based on divide & conquer technique



if $L[0] < R[0]$ swap $A[0] = L[0]$

$i++$, ~~$j++$~~ , $k++$

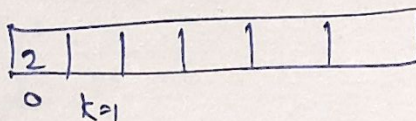
$L[1] > R[0]$ swap $A[1] = R[0]$

$j++$, $k++$

for this example.

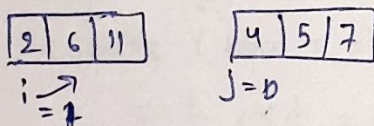
①

$2 < 4 \Rightarrow$
 $L \quad R$

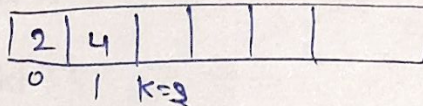


$j++$

②

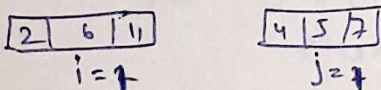


$6 > 4 \Rightarrow$

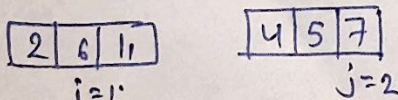
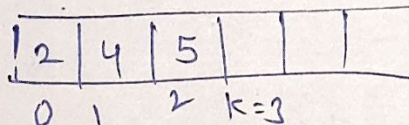


$j++$

③



$6 > 5$



$i++$

④ $6 < 7$

2	6	11
---	---	----

$i=2$

4	5	7
---	---	---

$j=2$

2	4	5	6		
---	---	---	---	--	--

⑤ $7 < 11$

2	6	11	∞
---	---	----	---

$i=2$

4	5	7	∞
---	---	---	---

$j=3$

2	4	5	6	7	
---	---	---	---	---	--

⑥ $11 < ∞$

2	6	11	∞
---	---	----	---

$i=3$

4	5	7	∞
---	---	---	---

$j=3$

2	4	5	6	7	11
---	---	---	---	---	----

Done

Algorithm

merge-sort(Arr, left, right)

1. if $\text{left} \geq \text{right}$, stop
2. $\text{mid} = \text{left} + (\text{right} - \text{left}) / 2$
3. merge-sort(Arr, left, mid)
4. merge-sort(Arr, mid+1, right)
5. merge(Arr, left, mid, right) // Combine

merge-sort(Arr, left, right)

1. Create two temp array, left & Right
2. Copy data
3. Maintain three pointer ($i \rightarrow \text{left}$, $j \rightarrow \text{Right}$, $k \rightarrow \text{main}$)
4. while
 - if $\text{left}[i] < \text{Right}[j]$
 $\text{Arr}[k] = \text{left}[i]$
 $i++$
 - else
 $\text{Arr}[k] = \text{Right}[j]$
 $j++$

k++;

5. Stop.

Java Code

```
public class Main {
```

```
    private static void merge (int arr[], int l, int m,  
                                int r) {
```

```
        int n1 = m - l + 1;
```

```
        int n2 = r - m;
```

```
        int[] L = new int[n1];
```

```
        int[] R = new int[n2];
```

```
        for (int i = 0; i < n1; i++)
```

```
        {  
            L[i] = arr[l + i];  
        }
```

```
        for (int j = 0; j < n2; j++)
```

```
        {  
            R[j] = arr[m + 1 + j];  
        }
```

Copy left &
right subarray

```
        int i = 0, j = 0;
```

```
        int k = l;
```

```
        while (i < n1 && j < n2) {
```

```
            if (L[i] <= R[j]) {
```

```
                arr[k] = L[i];
```

```
                i++;
```

```
            } else {
```

```
                arr[k] = R[j];
```

```
                j++;
```

```
            }
```



```

        k++;
    }
    while (i < n1) {
        arr[k] = L[i];
        i++;
        k++;
    }

```

```

    while (j < n2) {
        arr[k] = R[j];
        j++;
        k++;
    }

```

```

public static void mergeSort (int[] arr, int l, int r)
{
    if (l < r) {
        int m = l + (r - l) / 2;
        mergeSort (arr, l, m);
        mergeSort (arr, m + 1, r);
        merge (arr, l, m, r);
    }
}

```

```

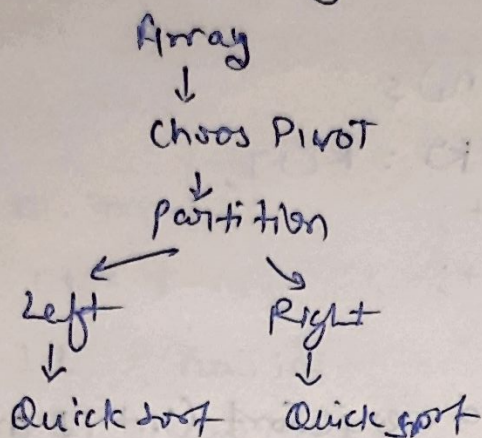
public static void main (String[] args) {
    int[] arr = {5, 2, 1, 3};
    mergeSort (arr, 0, arr.length - 1);
    System.out.print ("Sorted length array: ");
    for (int val : arr) {
        System.out.print (val + " ");
    }
}

```


* Quick Sort

Comparison based sorting algorithm based on divide & conquer method. But

→ we select a pivot element. then partition left & right sub array.



* Time Complexity

Best & Avg Case

$$= O(n \log n)$$

worst case = $O(n^2)$

* Space Complexity

Avg Case = $O(\log n)$

worst case = $O(n)$

Ex Array

8, 3, 5, 4, 7, 6, 1, 2

(pivot = 2 (for 1st step, last as pivot))

①

<u>1</u>	2	<u>8</u>	<u>3</u>	5	4	7	6
smaller		larger					
(sorted)		unsorted.					

② Now in Right side

8 3 5 4 7 6
pivot

3. 5 4 6 8 7
smaller larger

③ Now left side

3 5 4
pivot

3 4 5 sorted

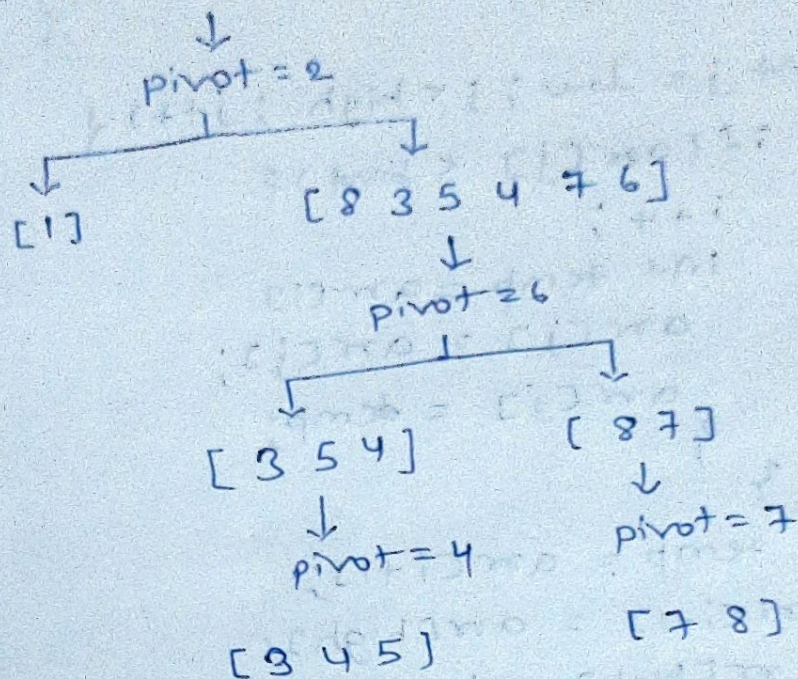
Now right side 8 7 pivot

7 8

Now Combine it

1 2 3 4 5 6 7 8 Done

[8 3 5 4 7 6 1 2]



Algorithm

QuickSort(arr, low, high)

if (low < high)

 pivot = partition(arr, low, high)

 QuickSort(arr, low, pivot - 1) // Left

 QuickSort(arr, pivot + 1, high) // Right

partition(arr, low, high)

 pivot = arr[high]

 i = low - 1

 for j from low to high - 1

 if arr[j] < pivot

 i++

 Swap arr[i] with arr[j]


```
swap arr[i+1] with arr[high]
return i+1
```

Java Code

```
public class main {
    private static int partition(int[] arr, low, int
                                high)
    {
        int pivot = arr[high];
        int i = (low - 1);

        for (int j = low; j < high; j++) {
            if (arr[j] < pivot) {
                i++;
                int temp = arr[i];
                arr[i] = arr[j];
                arr[j] = temp;
            }
        }

        int temp = arr[i+1];
        arr[i+1] = arr[high];
        arr[high] = temp;
        return i+1;
    }

    public static void quicksort(int arr[], int low,
                                  int high) {
        if (low < high) {
            int pivot = partition(arr, low, high);
            quicksort(arr, low, pivot - 1);
            quicksort(arr, pivot + 1, high);
        }
    }
}
```



```
public static void main(String[] args) {  
    int[] arr = {5, 2, 8, 1, 3};  
    int n = arr.length;  
    quicksort(arr, 0, n-1);  
    System.out.print("Sorted Array");  
    for(int val: arr) {  
        System.out.print(val + " ");  
    }  
}
```


* Strassen's Matrix Multiplication

As we know, if

$$A = \begin{bmatrix} a & b \\ c & d \end{bmatrix} \quad B = \begin{bmatrix} e & f \\ g & h \end{bmatrix} \quad \& \quad C = A \times B$$

$$\text{then } C = \begin{bmatrix} ae+bg & af+bh \\ ce+dg & cf+dh \end{bmatrix}$$

Total 8 multiplication.

$$\text{Time Complexity} = n \times n^2 = n^3$$

$$T(n) = O(n^3)$$

Strassen uses Divide & Conquer technique to reduce number of multiplication & make it efficient.

$$\text{So let } A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix} \quad B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$$

$$\& \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}$$

formulas

$$M_1 = (A_{11} + A_{22})(B_{11} + B_{22})$$

$$M_2 = B_{11}(A_{21} + A_{22})$$

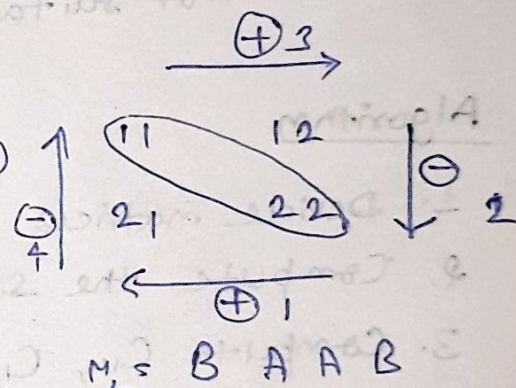
$$M_3 = A_{11}(B_{12} - B_{22})$$

$$M_4 = A_{22}(B_{11} + B_{12})$$

$$M_5 = B_{22}(A_{21} - A_{11})$$

$$M_6 = (B_{11} + B_{12})(A_{21} - A_{22})$$

$$M_7 = (B_{21} + B_{22})(A_{12} - A_{22})$$



So $C_{11} = m_1 + m_4 - m_5 + m_7$

$C_{12} = m_3 + m_5$

$C_{21} = m_2 + m_4$

$C_{22} = m_1 + m_3 - m_2 + m_6$

It required only 7 multiplication.

Time Complexity

for 8 multi. $T(n) = 8T(n/2) + O(n^2) = O(n^3)$

for 7 multiplication

$T(n) = 7T(n/2) + O(n^2) = O(n^{\log 7})$

$O(n^{\log 7}) < O(n^3)$

Notes

- It requires more addition/subtraction
- It is more complicated & requires extra memory
- Not suitable for small matrices.

Algorithm

1. Divide matrices A & B into four equal-sized submatrices
2. Compute the seven Strassen products m_1 to m_7
3. Compute C_{11} , C_{12} , C_{21} & C_{22} using these products
4. Combine the four resulting submatrices to form matrix C.
5. Return C.

a = number of recursive subproblem

b = size of each subpart

f(n) = recursive call work

Note

$T(n) = aT(n/b) + f(n)$